Linux Troubleshooting — Scenario #2: Website Is Down

DevOps on-call runbook • Syed Aftab • nginx on Ubuntu • Reference for real incidents & interviews

The alert (9:05 AM): web01 — users get an error loading the company site. It worked last night.
Root cause found: a one-character config mistake (missing `;`) applied with `restart`. nginx stopped, then refused to start on the broken config.

A broken config after a "small change" is one of the most common causes of web outages. This runbook is the exact flow used to find and fix it — and the habit that prevents it.

The methodology (same 6-phase flow as Scenario #1)

#	Phase	Command	Goal
1	Confirm	`curl -I http://localhost`	Is the site really down? What kind of failure?
2	Locate	`systemctl status nginx`	Is the service running or failed? Any clue in the logs?
3	Identify	`sudo nginx -t` `grep -n "listen" <file>`	Exact file + line number, and what is wrong on it
4	Fix safely	backup → `nano +22` → `nginx -t`	Repair without making it worse
5	Verify	`status`, `ss -tulpn`, `curl`	Prove it is running, listening, and serving 200
6	Prevent	`nginx -t && systemctl reload nginx`	A bad config can never take the site down again

SETUP Recreating the incident in a lab

```
sudo apt update
sudo apt install -y nginx
sudo systemctl start nginx
curl -I http://localhost        # healthy baseline: HTTP/1.1 200 OK
```

Then the "overnight change" that broke production:

```
sudo sed -i 's/listen 80 default_server;/listen 80 default_server/' \
     /etc/nginx/sites-enabled/default && sudo systemctl restart nginx
```

Part	Meaning
`sed`	**s**tream **ed**itor — finds and replaces text in a file without opening an editor
`-i`	edit the file **in place** (save the change). Without it, sed only prints the changed text and the file stays the same
`'s/old/new/'`	substitute: replace *old* with *new*. The `/` characters just separate the parts
`listen 80 default_server;`	old text — the real line, ending in `;`
`listen 80 default_server`	new text — same line with the `;` removed
`&&`	run the next command only if the previous one succeeded
`systemctl restart nginx`	stop nginx, then start it — it stopped fine, then failed to start

Tip: run sed **without** `-i` first to preview. Use `sed -i.bak` to keep an automatic backup — but never leave .bak files inside `sites-enabled/` (nginx loads every file in that folder).

STEP 1 Confirm — curl -I http://localhost

```
curl -I http://localhost
```

Part	Meaning
`curl`	send a web request from the command line (a browser without the screen)
`-I`	ask for the response **headers only** (status code, server) — not the whole page
`http://localhost`	this same machine, port 80 (the default http port)

Our result:

```
curl: (7) Failed to connect to localhost port 80 after 9 ms: Could not connect to server
```

Result	What it means
`HTTP/1.1 200 OK`	site works — problem is elsewhere (DNS, firewall, user side)
`curl: (7) ... Could not connect`	**nothing is listening on port 80** — the web server is down (our case)
`502 / 503`	nginx is up, but the application behind it is broken

Lesson: always read the error message. The failed restart already printed where to look next: `systemctl status nginx` and `journalctl -xeu nginx.service`.

STEP 2 Locate — systemctl status nginx

```
systemctl status nginx
```

Part	Meaning
`systemctl`	the tool that controls services (systemd)
`status`	is it running, since when, and the last few log lines
`nginx`	the service name

How we read the output:

Line	What it told us
`Loaded: ... enabled`	service exists and starts on boot — fine
`Active: failed (Result: exit-code)`	**nginx is dead** — it tried to start and exited with an error
`Duration: 2d 22h`	it ran fine for ~3 days, so something **changed**
`ExecStartPre=/usr/sbin/nginx -t ... status=1/FAILURE`	systemd runs a config test before starting. The test failed, so nginx never started
`[emerg] invalid parameter "listen" in /etc/nginx/sites-...`	**the clue** — emerg = most serious level; a `listen` line is wrong. Text was cut off (>)
`configuration file ... test failed`	confirms a **config problem** — not a crash, port conflict, or disk issue

Red herring: the yellow *"unit file ... changed on disk. Run 'systemctl daemon-reload'"* warning was **not** the cause. It only means systemd's own service file was updated (e.g. by a package install) and not re-read yet. Note it, don't chase it. Cleared later with `sudo systemctl daemon-reload`.

STEP 3 Identify — exact file, line, and mistake

```
sudo nginx -t
```

Part	Meaning
`nginx`	the nginx program itself (not the service)
`-t`	**test** the configuration and report errors — starts nothing, safe to run on a live server any time
`sudo`	needed to read all config files

```
[emerg] invalid parameter "listen" in /etc/nginx/sites-enabled/default:23
nginx: configuration file /etc/nginx/nginx.conf test failed
```

Full path and line number: `/etc/nginx/sites-enabled/default`, **line 23**. Next, look at the lines before touching anything:

```
grep -n "listen" /etc/nginx/sites-enabled/default
```

Part	Meaning
`grep`	search a file for text. Needs two things: **what** to search for, and **where**
`-n`	show the **line number** next to each matching line
`"listen"`	the word to find — taken straight from the error message
`cat -n file`	(alternative) show the **whole** file with line numbers

```
22:     listen 80 default_server          <-- missing ;
23:     listen [::]:80 default_server;    <-- nginx complained here
27:     # listen 443 ssl default_server;  <-- comment (#), ignored
80:#    listen 80;                        <-- comment, ignored
```

Key lesson — the error line can be one line AFTER the real mistake. nginx does not read line by line; it reads word by word until a `;`. A newline is just a space; only `;` ends a setting. With line 22 unfinished, nginx read:
`listen 80 default_server listen [::]:80 default_server;`
and failed at the second `listen` — which sits on line 23. **Rule:** if the reported line looks perfect, check the line above for a missing `;`, `}` or `"`.

What the two listen lines mean:

Part	Meaning
`listen 80`	accept connections on port 80, on **every IPv4 address** (same as `0.0.0.0:80`)
`listen [::]:80`	same for **IPv6**. `::` = all zeros = any IPv6 address. Brackets separate the address from the port, because IPv6 addresses contain colons
`default_server`	catch-all: answers any request that doesn't match another site's name

STEP 4 Fix safely — backup, edit, test

```
sudo cp /etc/nginx/sites-enabled/default ~/default.bak    # 1. backup
sudo nano +22 /etc/nginx/sites-enabled/default           # 2. add the ; on line 22
sudo nginx -t                                            # 3. test before applying
```

Part	Meaning
`cp src ~/default.bak`	copy the original to your home folder. If the fix goes wrong, copy it back. Keep it **outside** sites-enabled
`nano`	simple terminal text editor. Save: `Ctrl+O` then Enter. Exit: `Ctrl+X`
`+22`	open the file with the cursor already on line 22
`nginx -t`	must show *syntax is ok* and *test is successful* before anything is restarted

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

STEP 5 Verify — running, listening, serving

```
sudo systemctl start nginx        # service was DOWN, so start (not reload)
systemctl status nginx
sudo ss -tulpn | grep :80
curl -I http://localhost
```

Part	Meaning
`ss`	**s**ocket **s**tatistics — network connections and listening ports
`-t / -u`	TCP / UDP
`-l`	only **listening** ports (waiting for visitors)
`-p`	show the **process** that owns each port (needs sudo)
`-n`	show numbers (80) instead of names (http)
`| grep :80`	keep only port 80 lines. Use `:80` not `80` — plain 80 also matched `127.0.0.1:38077`

Lesson: `reload` only works on a **running** service. Our first attempt gave *"nginx.service is not active, cannot reload"*. Service down → `start`. Service running + config changed → `nginx -t && reload`.

Result: `Active: active (running)`; master + worker processes present; listening on `0.0.0.0:80` and `[::]:80`; `curl -I` returned **HTTP/1.1 200 OK**. Incident resolved.

STEP 6 Prevent — never let a bad config take the site down

The real root cause: the missing `;` was the trigger, but the outage happened because the change was applied with `restart`.

Command	What it does	If the new config is broken
`restart`	stops nginx, then starts it	stop works, start fails → **site down**
`reload`	running nginx re-reads its config	nginx rejects it and keeps running on the old config → **site stays up**

1. Always apply nginx changes this way:

```
sudo nginx -t && sudo systemctl reload nginx
```

2. Proven in the lab: we broke line 22 again and ran the safe command. `nginx -t` failed, `&&` stopped the chain, reload never ran, and `systemctl status` still showed **active (running)**. nginx keeps the good config in **memory**; the broken file only matters on restart, reload, or **reboot** — so the file must still be fixed right away.

3. Clear warnings, don't ignore them: `sudo systemctl daemon-reload`

4. Know before users do — a monitoring tool (Uptime Kuma, Prometheus blackbox exporter, Datadog, CloudWatch Synthetics) checks every minute and alerts on-call. Underneath, it is this check:

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost      # prints 200 when healthy
```

Part	Meaning
`-s`	silent: no progress bar
`-o /dev/null`	throw away the page body (curl option — only the body is discarded)
`-w "%{http_code}\n"`	print only the status code, plus a newline so it doesn't stick to the prompt
`&> /dev/null` (wrong here)	a **shell** redirect — discards **all** output, including the 200. It also left `-o` with no filename, so `-o` took `-w` as one

REF Full command reference

Command	What	Why here
`curl -I http://localhost`	fetch response headers	confirm the site is down / back up
`systemctl status nginx`	service state + recent logs	locate: running or failed, first clue
`journalctl -xeu nginx.service`	full service logs (x = explanations, e = jump to end, u = unit)	read the full error when status cuts it off
`sudo nginx -t`	test config, start nothing	exact file + line of the error
`grep -n "listen" file`	matching lines with numbers	see the broken line and its neighbours
`cp file ~/file.bak`	backup	safe rollback point before editing
`nano +22 file`	edit at line 22	apply the fix
`sed -i 's/a/b/' file`	find/replace in place	how the bad change was made
`systemctl start nginx`	start a stopped service	bring the site back after the fix
`ss -tulpn | grep :80`	who is listening on port 80	prove nginx owns the port
`nginx -t && systemctl reload nginx`	test, then zero-downtime apply	prevention: bad configs never go live
`systemctl daemon-reload`	systemd re-reads unit files	clear the "changed on disk" warning

Interview answer (say this)

"Site down? I confirm it myself with curl. A connection refused means nothing is listening, so I check systemctl status and see the service failed its config test. I run nginx -t to get the exact file and line — remembering the real mistake is often the line before, like a missing semicolon. I back up the file, fix it, test again, start the service, and verify with status, ss, and curl. Then I prevent a repeat: config changes only go out as nginx -t && systemctl reload, so a bad config is rejected and the site stays up — plus monitoring on the HTTP status code."